WEEK 1

Introduction to Object-Oriented Programming

From Procedural Thinking to Objects — Python 3.x

UYG203 · Object Oriented Programming · Software Development

14-Week Roadmap

Where today's session fits into the semester

W1 Intro to OOP	**W2** Classes & Objects	**W3** Attributes & Methods	**W4** Encapsulation	**W5** Inheritance	**W6** Polymorphism	**W7** Abstraction
W8 Midterm	**W9** Composition	**W10** Exceptions	**W11** File Handling	**W12** Modules & Testing	**W13** Project Dev.	**W14** Presentations

Learning Outcomes — Week 1

By the end of today, you will be able to:

1. Explain the difference between procedural and object-oriented programming, and describe at least three concrete limitations of the procedural style
2. Define the terms class, object, attribute, and method in your own words
3. Name and briefly describe the four core OOP principles we will build toward this semester
4. Configure a working Python 3.x + VS Code development environment
5. Write and run a Python script from both the terminal and VS Code

Today's Agenda

Two hours of theory, two hours of hands-on lab

1	**Python Review (15 min)**	A fast refresher on syntax we'll rely on all semester
2	**Procedural Programming, Revisited (25 min)**	What it is, and a case study showing where it strains
3	**Introducing OOP (35 min)**	Objects, classes, and a first look at the four pillars
4	**Why OOP? (15 min)**	Modularity, reusability, maintainability, real-world modeling
5	**Environment Setup (20 min)**	Python + VS Code, verified and ready for the lab
6	**Lab: First Python Program (30 min)**	Hands-on practice, procedural exercises + reflection

Quick Python Recap

You've already used all of this in Introduction to Programming — let's warm up

```
# variables & types
name = "Ada"          # str
age = 20               # int
gpa = 3.75             # float
is_active = True       # bool

# f-strings
print(f"{name} is {age} years old")
```

```
# conditionals & loops
for grade in [92, 68, 74]:
    if grade >= 70:
        print("Pass")
    else:
        print("Fail")
```

Why this matters today

- Every OOP concept we introduce this semester builds on these exact fundamentals
- Classes contain functions (called methods) that use this same syntax internally
- If any of this feels shaky, flag it during the lab — we'll reinforce it before moving to classes next week

What Is Procedural Programming?

A programming paradigm where a program is built as a sequence of instructions and function (procedure) calls that operate on data — the data and the logic that acts on it are kept separate.

Key characteristics

- The program is organized around a top-to-bottom flow of execution — main() calls functions in order
- Data is typically stored in simple structures: variables, lists, dictionaries, tuples
- Functions receive data as parameters and return results — they don't “belong” to the data
- This is exactly the style used in your Introduction to Programming course, and it's a completely valid way to write small, linear programs

A Procedural Example, Line by Line

```
student_name = "Ada"
student_grade = 92

def print_report(name, grade):
    print(f"{name}: {grade}")

def letter_grade(grade):
    if grade >= 90:
        return "AA"
    return "Other"

print_report(student_name, student_grade)
print(letter_grade(student_grade))
```

What's happening here

- student_name and student_grade are plain variables — nothing ties them together except that we, the programmer, remember to pass both
- print_report() and letter_grade() are standalone functions — they could just as easily operate on a completely different student's data
- Nothing in the code stops you from calling letter_grade(student_name) by mistake — the data has no “self-knowledge” of what it represents
- This works fine for one student. Let's see what happens at scale.

CASE STUDY

Scaling Up: From 1 Student to 30

Let's grow the program above into something closer to a real classroom, and watch where it starts to strain.

```
students = [
    ("Ada", 92),
    ("Mert", 84),
    ("Elif", 58),
    # ... 27 more students
]

for name, grade in students:
    print_report(name, grade)
    print(letter_grade(grade))
```

Manageable... for now

- Tuples keep (name, grade) paired, and a loop avoids copy-pasting the function calls
- But this only works because every student has exactly two pieces of data
- The moment we need more information per student, the tuple stops being enough

New Requirement: Add Attendance Tracking

The registrar's office now wants attendance rate tracked, with a hard rule: below 70% attendance is an automatic fail, regardless of grade.

```
students = [
    ("Ada", 92, 0.95),
    ("Mert", 84, 0.65),   # fails on attendance!
    ("Elif", 58, 0.88),
]

def passes(name, grade, attendance):
    if attendance < 0.70:
        return False
    return grade >= 60
```

What just happened?

- ⚠ Every tuple in the list had to be updated to add the third field
- ⚠ Every function that unpacks a student tuple (print_report, letter_grade, and now passes) needs its signature checked or rewritten
- ⚠ It's easy to unpack fields in the wrong order (name, attendance, grade) with no error — tuples don't enforce meaning, only position

CASE STUDY, CONTINUED

The Copy-Paste Problem

Now the course also needs to track Instructors and Teaching Assistants — two more “things” that also have names, and their own rules.

```
def print_student_report(name, grade, attendance): ...
def print_instructor_report(name, courses, office): ...
def print_ta_report(name, hours, supervisor): ...

# Three near-identical functions, three near-identical
# data shapes, and zero shared structure between them.
```

The pattern to notice

- ⚠ Every new “kind of thing” in the system means another parallel set of tuples and functions — this is the copy-paste explosion
- ⚠ There is no single place that represents “what a Student is” — that knowledge is scattered across every function that touches student data
- ⚠ This is precisely the gap that a class is designed to close: one definition, in one place, for what a Student is and does

Five Pain Points of Procedural Code at Scale

No binding between data and behavior	Nothing stops passing the wrong tuple to the wrong function
Fragile position-based data	Tuples/lists rely on field order, not names — easy to shuffle by accident
Duplicated logic per entity type	Student, Instructor, TA each need their own nearly-identical functions
Hard to extend safely	Adding one field means touching every function that unpacks that data
No natural place for validation	Nothing enforces that attendance stays between 0 and 1, for example

There's a Better Way to Organize This

Object-Oriented Programming ties data and behavior together, in one place, per “kind of thing.”

What Is Object-Oriented Programming?

A programming paradigm that organizes software around objects — self-contained units that bundle together data (attributes) and the behavior that operates on that data (methods).

The core shift in thinking

- Procedural: “What sequence of steps solves this problem?”
- Object-oriented: “What kinds of things exist in this problem, and what do they know and do?”
- A Student object doesn't just hold a name and grade — it can carry a method like calculate_gpa() that belongs to it directly

The Core Idea: Object = State + Behavior

State (Attributes)

The data an object holds — its current values, at this moment in time.

```
self.name = "Ada"
self.grade = 92
self.attendance = 0.95
```

Think: nouns — the things you'd list if describing a student on paper.

Behavior (Methods)

The actions an object can perform — usually using or changing its own state.

```
def is_passing(self):
    if self.attendance < 0.70:
        return False
    return self.grade >= 60
```

Think: verbs — the things a student can do, or that can be done to them.

Real-World Analogy: A Car

Before we touch Python syntax, let's build the intuition in plain language

Attributes — what a car has

- Color, brand, model, current speed, fuel level, number of doors
- Each of these can be different for every car, even cars of the same model

Methods — what a car can do

- accelerate(), brake(), refuel(), honk_horn()
- These actions typically read or change the car's own attributes — braking changes speed, refueling changes fuel level

From Analogy to Code: The Blueprint Concept

A Class Is a Blueprint

It defines what attributes and methods every car will have — but it isn't a car itself. Nobody can drive a blueprint.

Compare to the factory that designs a car model once, then manufactures many individual cars from that one design.

An Object Is an Instance

A specific car built from the blueprint — with its own color, its own current fuel level.

```
# next week's syntax, previewed:
my_car = Car("red", "Toyota")
your_car = Car("blue", "Fiat")

my_car.accelerate()
```

Class vs. Object — Don't Confuse the Two

Term	What it is	How often	Example
Class	Definition / template	Written once	`Student`
Object	A specific instance	Created many times	`ada = Student("Ada", 92)`

Common analogy: “Class” is to “Cookie cutter” as “Object” is to “Cookie.” One cutter, many cookies — each can have its own icing (state).

One Class, Many Objects

Each object has the same structure, but its own independent state

class Student

Ada object

`grade = 92`

Mert object

`grade = 84`

Elif object

`grade = 58`

Notice

- All three share the same attributes (name, grade) and methods, because they all come from the Student class
- But each object's values are independent — changing Ada's grade does not affect Mert's
- This is exactly what tuples could not guarantee for us in Part 1

The Four Pillars of OOP

We'll dedicate a full week to each — today is a first preview



Encapsulation

Week 4



Inheritance

Week 5



Polymorphism

Week 6



Abstraction

Week 7

Each pillar solves a specific problem we ran into in Part 1's case study — we'll connect the dots explicitly each week.

PILLAR PREVIEW 1 OF 4

Encapsulation



What it means

- Bundling data and the methods that operate on it inside one object, and controlling access to that data from the outside
- Prevents attendance from being set to 1.5, or a grade from being set to "banana", by validating changes through methods instead of direct access
- Directly solves Pain Point 5 from Part 1: “no natural place for validation”
- Covered in depth in Week 4, including Python's public / protected / private conventions and the @property decorator

PILLAR PREVIEW 2 OF 4

Inheritance



What it means

- A new class can be built on top of an existing one, automatically getting its attributes and methods, then adding or overriding only what's different
- A GraduateStudent class could inherit everything from Student and add a thesis_title attribute — without rewriting name, grade, or is_passing()
- Directly solves Pain Point 3 from Part 1: “duplicated logic per entity type”
- Covered in depth in Week 5, including the super() function and building class hierarchies

PILLAR PREVIEW 3 OF 4

Polymorphism



What it means

- Different classes can implement the same method name in their own way, and calling code doesn't need to know which exact class it's dealing with
- print_report() could work identically whether called on a Student, Instructor, or TA object, each printing itself appropriately
- Removes the need for the copy-pasted print_student_report / print_instructor_report / print_ta_report functions from Part 1
- Covered in depth in Week 6, through method overriding and Python's duck typing

PILLAR PREVIEW 4 OF 4

Abstraction



What it means

- Exposing only what a user of a class needs to know, and hiding the implementation details behind a simple interface
- You call my_car.accelerate() without needing to know how the engine actually manages fuel injection internally
- Makes large systems easier to reason about — you interact with a small, well-defined surface, not the full complexity underneath
- Covered in depth in Week 7, using Python's abc module to define abstract classes and required methods

Procedural vs. Object-Oriented — Side by Side

Aspect	Procedural	Object-Oriented
Organizing unit	Functions operating on data	Objects bundling data + behavior
Data & logic	Kept separate	Bundled together
Reusability	Copy-paste per entity type	Inherit and extend
Validation	Manual, easy to forget	Enforced inside the object
Scaling to new features	Touches many functions	Often touches one class

Why Object-Oriented Programming? (1/2)



Modularity

Large systems get broken into self-contained units. Each class can be understood, tested, and debugged largely on its own, without holding the entire program in your head at once.



Reusability

Through inheritance, new classes can build on existing ones instead of starting from scratch. A well-designed Student class becomes the foundation for GraduateStudent, ExchangeStudent, and more.

Why Object-Oriented Programming? (2/2)



Maintainability

Fixing a bug in how attendance is validated means editing the Student class in one place — every part of the program that uses a Student object gets the fix automatically.



Real-World Modeling

Objects map naturally onto the entities we're already thinking about: Student, Course, Instructor, BankAccount, Vehicle. Code structure ends up mirroring the problem domain, which makes it easier to discuss and design.

Python as an Object-Oriented Language

A few facts worth knowing

- Everything in Python is technically an object already — even an int or a str has methods, like "hello".upper()
- Python supports OOP but doesn't force it — you can mix procedural functions and classes freely, and we'll do exactly that this semester
- Classes are defined with the class keyword, and every instance method's first parameter is conventionally named self
- Python does not enforce strict access control the way some languages do — encapsulation in Python relies on naming conventions (Week 4) more than hard restrictions

Step 1 — Install Python 3.x

Instructions

- Go to python.org/downloads and download the latest Python 3.x installer for your OS
- Windows: during setup, check “Add python.exe to PATH” — this is the single most common setup mistake
- macOS: consider installing via python.org directly, or Homebrew (brew install python3) if you already use it
- Linux: most distributions ship Python 3 already — confirm the version is 3.10 or newer

Step 2 — Install VS Code & the Python Extension

Instructions

- Download VS Code from code.visualstudio.com — it's free and works the same on Windows, macOS, and Linux
- Open the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X) and install “Python” by Microsoft
- This extension adds IntelliSense (autocomplete), linting (error highlighting), and a built-in debugger — all of which we'll use starting next week
- Optional but recommended: install the “Jupyter” extension too, since our weekly lab notebooks run inside VS Code

Step 3 — Verify Everything Works

```
$ python --version
Python 3.12.4

$ python
>>> print("Hello, OOP!")
Hello, OOP!
>>> exit()
```

If something's wrong

- ⚠ “command not found” usually means PATH wasn't set — reinstall and check the PATH option
- ⚠ Multiple Python versions installed? Try python3 --version instead of python --version
- ⚠ Ask during the lab — we'll fix environments before moving on

Once python --version prints 3.x and the print() statement runs without errors, your environment is ready for the lab.

This Week's Lab

Environment Setup & Your First Python Program

- Install and verify Python 3.x and VS Code
- Create your first .py file and run it from the terminal
- Write a script using variables, input(), and print()
- Complete the procedural review exercises (Jupyter notebook)
- Rewrite one review exercise using a function instead of inline code

Key Takeaways

- Procedural code separates data and functions; at scale, this creates duplicated logic, fragile data structures, and no natural home for validation
- OOP bundles state (attributes) and behavior (methods) into objects, modeled after real-world things
- A class is a blueprint; an object is one instance built from that blueprint, with its own independent state
- Four pillars — encapsulation, inheritance, polymorphism, abstraction — each solve a specific pain point we identified today
- Your Python + VS Code environment is the foundation for every lab this semester

Next Week: Classes and Objects

Week 2

Classes and Objects

- Defining classes with the class keyword and the __init__() constructor
- Understanding self and how objects come into existence
- Lab: rebuild today's Student example as an actual class — and compare it directly to Part 1's procedural version
- Developing Student, Book, and Product classes

Questions?

Before next class: finish the Week 1 lab notebook, and be ready to see it rebuilt as a class next week.